

Ziyue Fang

Mobile: (+86) 18856978796 | Email: fangziyue0623@outlook.com

EDUCATION

Hefei University of Technology | Hefei, China

Sep 2022 – Jun 2026

- Bachelor of Electronic Information Engineering • GPA 3.6/4.3

The University of Hong Kong | Hong Kong, China

Sep 2026 – Present

- Master of Robotics Intelligent Systems

INTERNSHIP EXPERIENCE

iFLYTEK|Hefei China Embedded Software Development Engineer Intern

Aprs.2025-June.2025

- Built a domain knowledge-graph GraphRAG service with a supporting graph retrieval pipeline.
- Developed an enterprise document Q&A Agent, building the document processing and Q&A service with Flask that supports PDF/Word upload, knowledge base management, intelligent retrieval, and RESTful API calls.
- Built the domain knowledge graph (GraphRAG): modeled key entities, attributes, and relationships from documents into a graph structure using Neo4j; implemented entity locating, relationship-path expansion, and neighborhood-subgraph queries with Cypher to provide structured knowledge support for complex relational and cross-document questions.
- Designed a hybrid retrieval pipeline combining vector semantic retrieval with graph-based retrieval: vectorized text chunks with a BGE embedding model and recalled relevant passages via top-K similarity search; merged Neo4j graph retrieval results to complete entity context and relationships.
- Unified vector-recall fragments and graph query results into grounded context injected into a Qwen LLM, forming a closed-loop GraphRAG pipeline of graph retrieval → vector retrieval → LLM generation; used prompt engineering to constrain the model to answer only from retrieved evidence, reducing hallucination risk.
- Parsed unstructured PDF and Word text with PyMuPDF and python-docx; applied sliding-window chunking (1,000-character chunks with 200-character overlap) to reduce semantic truncation in long documents and improve retrieval coverage.
- Implemented a retrieval router that prioritizes the public knowledge base and supplements with uploaded documents; returned matched text, knowledge sources, and graph associations to improve answer explainability and traceability.
- Strengthened engineering practice: added upload file-type validation, MD5 file fingerprinting, thread-safe knowledge base reads/writes, scheduled cleanup of expired data, unified logging, and exception handling, improving service stability and maintainability.

PROJECT EXPERIENCE

LangGraph-Based Multi-Agent Software Engineering Assistant

- Designed a multi-agent collaboration architecture on LangGraph's StateGraph, building five agent types (Supervisor, Explorer, Coder, Reviewer, Executor); orchestrated via a state machine to achieve task decomposition, dynamic routing, result aggregation, and conditional transitions.
- Implemented a Supervisor orchestration layer: identified user intent and decomposed it into subtasks such as code retrieval, development, review, and execution verification; selected the target agent based on task state to form a controllable agent workflow.
- Built an Explorer agent integrating local retrieval tools such as ripgrep to search repository directory structures, function definitions, dependencies, and call relationships and aggregate context, providing grounded context for the code generation phase.
- Designed a Coder-Reviewer iterative feedback mechanism: the Coder generated or modified code based on context while the Reviewer evaluated functional correctness, code style, and security; on failure, structured feedback was generated and routed back to the Coder, with a maximum iteration limit to avoid ineffective loops.
- Built an Executor agent that ran test cases in an isolated execution environment, captured standard output and exception stacks, and fed verification results back to the Supervisor, completing a closed loop of generate → review → test → fix.
- Persisted LangGraph checkpoints and session state with Redis to support task interruption and recovery; stored task execution traces and audit logs in PostgreSQL; built a command-line interface with Rich and prompt_toolkit.

RAG Local Knowledge Base Intelligent Q&A System

- Built a localized document knowledge base Q&A system covering a complete RAG pipeline including document parsing, index building, hybrid retrieval, context assembly, LLM generation, and quality evaluation.
- Designed a hybrid retrieval mechanism: combined BM25 keyword retrieval with vector semantic retrieval based on pgvector and bge-m3 for dual-channel recall, fused rankings with Reciprocal Rank Fusion (RRF), and applied dynamic top-K adaptive truncation to select high-quality context.
- Implemented a query understanding pipeline: applied query rewriting for multi-turn dialogues to resolve references; performed query decomposition on compound questions into independently retrievable sub-queries, which the LLM aggregated into final answers.
- Adopted a parent-child two-level chunking strategy: built vector indexes with fine-grained chunks to ensure retrieval precision, then traced back to parent chunks to complete generation context, balancing retrieval relevance with contextual completeness.
- Built an LLM-as-a-Judge evaluation process that automatically assessed whether retrieved context covered the question; integrated RAGAS to evaluate RAG generation quality across dimensions such as faithfulness, answer relevancy, and context precision.
- Provided a document-upload and token-streaming interface with Streamlit; orchestrated PostgreSQL and pgvector services with Docker Compose to enable one-click local deployment, environment isolation, and reproducible runs.

OTHER

Computer Skills: CPP, Python, Sql, verilog, vhdl, ROS2, URDF